# LAPORAN PRAKTIKUM

# ALGORITMA PEMROGRAMAN 2

# MODUL 10 – Pencarian Nilai Max/Min

SERLIN APRILIA

109082530032

KELAS S1IF-13-06

**PROGRAM STUDI TEKNIK INFORMATIKA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2026**

A. Dasar Teori

Dalam dunia pemrograman, pengolahan data merupakan hal yang sangat penting, terutama dalam mencari informasi tertentu dari sekumpulan data. Salah satu bentuk pengolahan data yang sering digunakan adalah pencarian nilai maksimum (max), minimum (min), dan rata-rata (mean). Nilai maksimum adalah nilai terbesar dalam suatu kumpulan data, sedangkan nilai minimum adalah nilai terkecil. Rata-rata merupakan nilai yang mewakili keseluruhan data yang diperoleh dengan menjumlahkan seluruh elemen kemudian dibagi dengan jumlah data.

Untuk melakukan proses tersebut, biasanya digunakan struktur data seperti array atau slice yang mampu menyimpan banyak data dengan tipe yang sama. Dalam bahasa pemrograman Go, array memiliki ukuran tetap, sedangkan slice lebih fleksibel karena ukurannya dapat berubah. Proses pencarian nilai minimum dan maksimum umumnya menggunakan metode linear search, yaitu teknik pencarian dengan cara membandingkan setiap elemen secara berurutan mulai dari indeks pertama hingga terakhir. Metode ini sederhana dan mudah diimplementasikan, meskipun memiliki kompleksitas waktu O(n), yang berarti waktu eksekusi bergantung pada jumlah data.

Selain itu, penggunaan fungsi atau subprogram seperti cariMin, cariMax, dan cariRata sangat membantu dalam membagi program menjadi bagian-bagian kecil yang memiliki tugas spesifik. Hal ini membuat program menjadi lebih modular, mudah dipahami, serta memudahkan dalam proses pengujian dan perawatan kode. Konsep lain yang digunakan adalah perulangan (looping) untuk mengakses setiap elemen dalam array, serta penggunaan input-output untuk menerima data dari pengguna dan menampilkan hasil ke layar. Dengan menggabungkan konsep-konsep tersebut, program dapat mengolah data secara sistematis dan efisien.

B. Guided

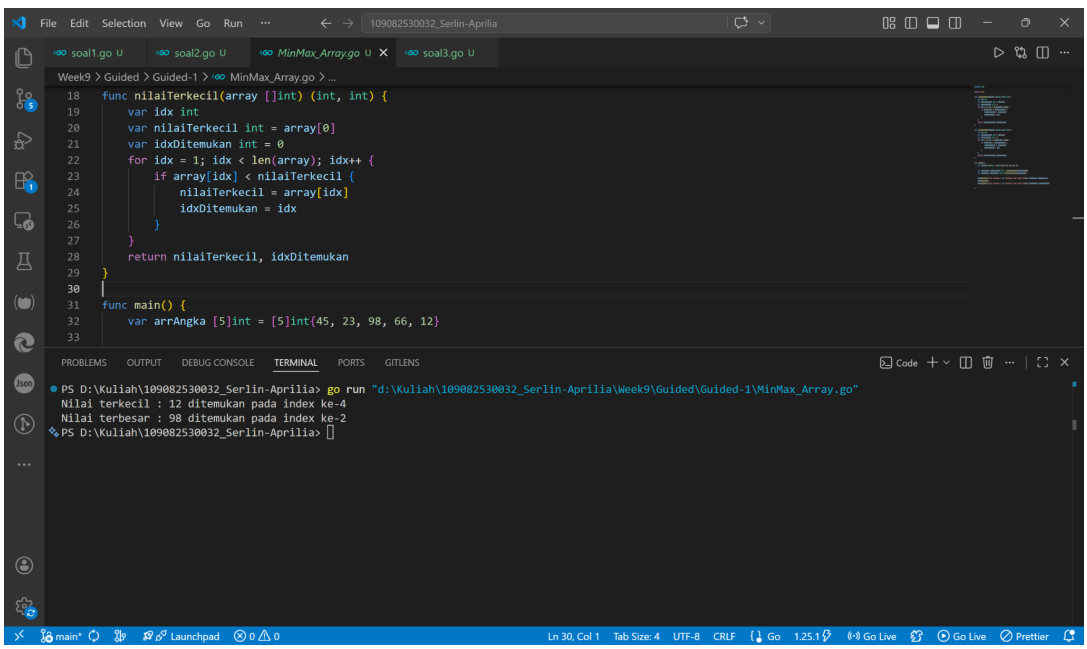
1. Minmax array

```
package main


import "fmt"
```

```
func nilaiTerbesar(array []int) (int, int) {
	var idx int
	var nilaiTerbesar int = array[0]
	var idxDitemukan int = 0
	for idx = 1; idx < len(array); idx++ {
		if array[idx] > nilaiTerbesar {
			nilaiTerbesar = array[idx]
			idxDitemukan = idx
		}
	}
	return nilaiTerbesar, idxDitemukan
}

func nilaiTerkecil(array []int) (int, int) {
	var idx int
	var nilaiTerkecil int = array[0]
	var idxDitemukan int = 0
	for idx = 1; idx < len(array); idx++ {
		if array[idx] < nilaiTerkecil {
			nilaiTerkecil = array[idx]
			idxDitemukan = idx
		}
	}
	return nilaiTerkecil, idxDitemukan
}

func main() {
	var arrAngka [5]int = [5]int{45, 23, 98, 66, 12}
```

```
    var nilaiMaks, idxNilaiMaks int = nilaiTerbesar(arrAngka[:])

    var nilaiMin, idxNilaiMin int = nilaiTerkecil(arrAngka[:])


    fmt.Printf("Nilai terkecil : %d ditemukan pada index ke-%d", nilaiMin, idxNilaiMin)

    fmt.Println()

    fmt.Printf("Nilai terbesar : %d ditemukan pada index ke-%d", nilaiMaks, idxNilaiMaks)


}
```

Output :




Kode tersebut merupakan program Go yang digunakan untuk mencari nilai terkecil dalam sebuah array beserta indeksnya. Fungsi nilaiTerkecil(array []int) (int, int) menerima parameter berupa slice/array bilangan bulat, lalu menginisialisasi variabel nilaiTerkecil dengan elemen pertama (array[0]) dan idxDitemukan dengan nilai 0 sebagai indeks awal. Selanjutnya dilakukan perulangan dari indeks ke-1 hingga akhir array untuk

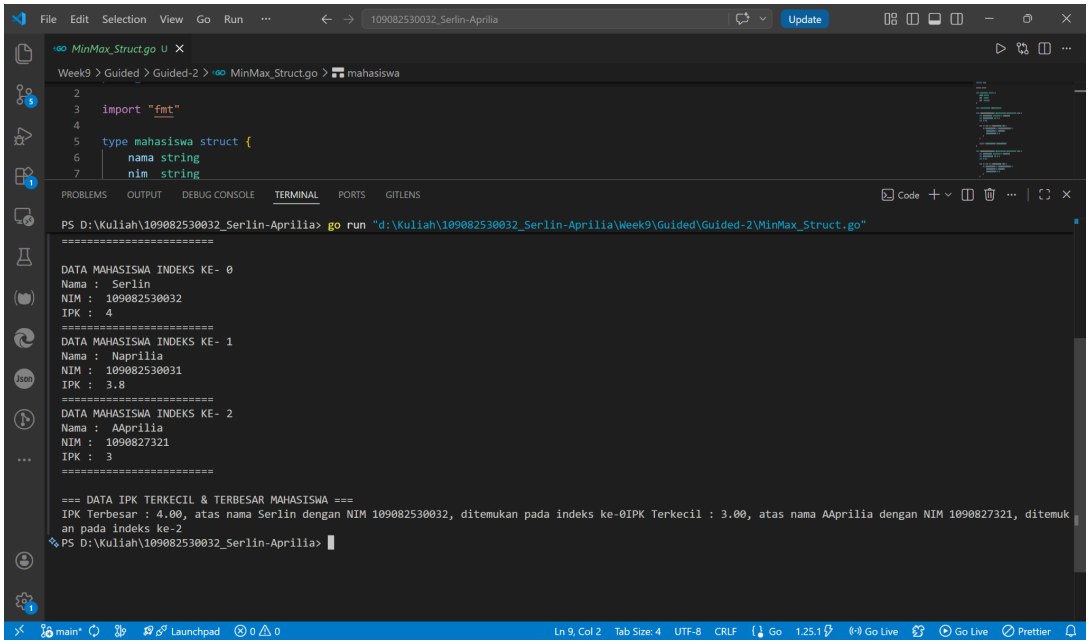
membandingkan setiap elemen dengan nilai terkecil saat ini. Jika ditemukan elemen yang lebih kecil, maka nilai tersebut akan menggantikan nilaiTerkecil dan indeksnya disimpan di idxDitemukan. Setelah perulangan selesai, fungsi mengembalikan dua nilai yaitu nilai terkecil dan posisi indeksnya. Pada fungsi main, dibuat array arrAngka berisi lima angka {45, 23, 98, 66, 12}, kemudian fungsi dipanggil (bersama fungsi lain untuk nilai terbesar yang tidak terlihat penuh di gambar) dan hasilnya ditampilkan ke terminal, sehingga menghasilkan output nilai terkecil 12 di indeks ke-4 dan nilai terbesar 98 di indeks ke-2.

2. Minmax struct

```
package main

import "fmt"

type mahasiswa struct {
      nama string
      nim  string
      IPK  float64
}

type arrMahasiswa [3]mahasiswa

func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
      var mhsTerbesar mahasiswa = array[0]
      var idxDitemukan int = 0
      var i int

      for i = 1; i < len(array); i++ {
            if array[i].IPK > mhsTerbesar.IPK {
                  mhsTerbesar = array[i]
                  idxDitemukan = i
            }
      }

      return mhsTerbesar, idxDitemukan
}

func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
      var mhsTerkecil mahasiswa = array[0]
      var idxDitemukan int = 0
      var i int

      for i = 1; i < len(array); i++ {
            if array[i].IPK < mhsTerkecil.IPK {
                  mhsTerkecil = array[i]
                  idxDitemukan = i
            }
```

```
	}
	return mhsTerkecil, idxDitemukan
}

func main() {
	var arrMhs arrMahasiswa
	var i, j int
	for i = 0; i < len(arrMhs); i++ {
		fmt.Println("DATA MAHASISWA INDEKS KE-", i)
		fmt.Print("Masukkan nama : ")
		fmt.Scan(&arrMhs[i].nama)
		fmt.Print("Masukkan NIM :")
		fmt.Scan(&arrMhs[i].nim)
		fmt.Print("Masukkan IPK : ")
		fmt.Scan(&arrMhs[i].IPK)
		fmt.Println("========================")
	}

	fmt.Println()
	for j = 0; j < len(arrMhs); j++ {
		fmt.Println("DATA MAHASISWA INDEKS KE-", j)
		fmt.Println("Nama : ", arrMhs[j].nama)
		fmt.Println("NIM : ", arrMhs[j].nim)
		fmt.Println("IPK : ", arrMhs[j].IPK)
		fmt.Println("========================")
	}
	fmt.Println()

	var mhsMaks, mhsMin mahasiswa
	var idxMaks, idxMin int

	mhsMaks, idxMaks = IPKTerbesar(arrMhs[:])
	mhsMin, idxMin = IPKTerkecil(arrMhs[:])

	fmt.Println("=== DATA IPK TERKECIL & TERBESAR
MAHASISWA ===")
	fmt.Printf("IPK Terbesar : %.2f, atas nama %s dengan
NIM %s, ditemukan pada indeks ke-%d", mhsMaks.IPK,
mhsMaks.nama, mhsMaks.nim, idxMaks)
	fmt.Printf("IPK Terkecil : %.2f, atas nama %s dengan
NIM %s, ditemukan pada indeks ke-%d", mhsMin.IPK,
mhsMin.nama, mhsMin.nim, idxMin)
}
```

Output :




Program tersebut merupakan implementasi pengolahan data mahasiswa menggunakan **struct dan array di bahasa Go**. Struct `mahasiswa` digunakan untuk merepresentasikan satu entitas data yang memiliki atribut `nama`, `nim`, dan `IPK`, kemudian disimpan dalam array bertipe `arrMahasiswa` dengan kapasitas tetap sebanyak 3 elemen. Pada bagian `main`, program melakukan input data untuk setiap indeks array, lalu menampilkannya kembali sebagai verifikasi data. Selanjutnya, dua fungsi yaitu `IPKTerbesar` dan `IPKTerkecil` digunakan sebagai subprogram untuk melakukan pencarian nilai maksimum dan minimum berdasarkan field `IPK`. Kedua fungsi ini bekerja dengan teknik **linear search**, yaitu menginisialisasi nilai awal dari elemen pertama array, kemudian melakukan iterasi untuk membandingkan setiap elemen berikutnya; jika ditemukan nilai yang lebih besar atau lebih kecil, maka data struct dan indeksnya diperbarui. Fungsi mengembalikan dua nilai sekaligus (multiple return), yaitu data mahasiswa dan indeks posisi dalam array. Hasil akhir program menampilkan mahasiswa dengan IPK tertinggi dan terendah lengkap dengan identitasnya (nama, NIM) serta posisi indeksnya dalam array, dengan format output menggunakan `fmt.Printf` untuk pengaturan tampilan angka desimal.

C. Unguided

1. Soal Unguided 1

1) Sebuah program digunakan untuk mendata berat anak kelinci yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat anak kelinci yang akan dijual.

   **Masukan** terdiri dari sekumpulan bilangan, yang mana bilangan pertama adalah bilangan bulat N yang menyatakan banyaknya anak kelinci yang akan ditimbang beratnya. Selanjutnya N bilangan riil berikutnya adalah berat dari anak kelinci yang akan dijual.

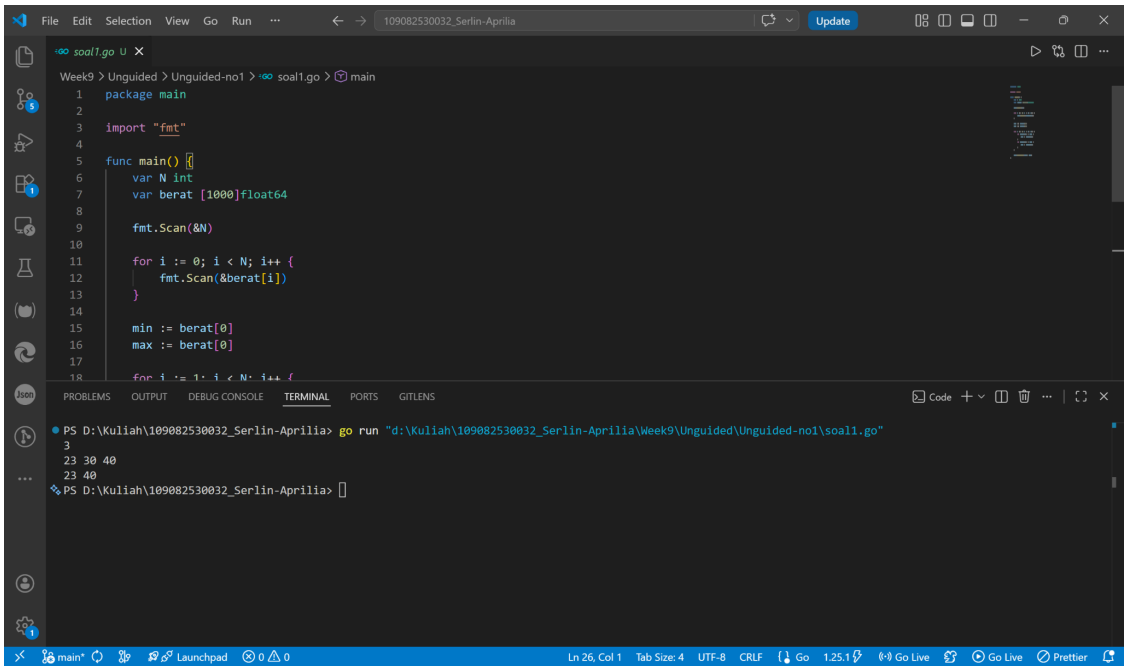
   **Keluaran** terdiri dari dua buah bilangan riil yang menyatakan berat kelinci terkecil dan terbesar.

Jawaban :

```
package main


import "fmt"


func main() {
      var N int
      var berat [1000]float64


      fmt.Scan(&N)


      for i := 0; i < N; i++ {
            fmt.Scan(&berat[i])
      }


      min := berat[0]
      max := berat[0]


      for i := 1; i < N; i++ {
            if berat[i] < min {
                  min = berat[i]
```

```
            }
            if berat[i] > max {
                max = berat[i]
            }
        }

        fmt.Println(min, max)
}
```

Output :




Penjelasan :

Program Go tersebut digunakan untuk mencari nilai minimum dan maksimum dari sekumpulan data berat yang dimasukkan oleh pengguna. Baris package main menunjukkan bahwa program ini adalah program utama yang dapat langsung dijalankan, sedangkan import "fmt" digunakan untuk mengakses fungsi input/output seperti Scan dan Println. Di dalam fungsi main, dideklarasikan variabel N untuk menyimpan jumlah data, serta array berat berkapasitas 1000 untuk menampung nilai bertipe float64. Program kemudian membaca nilai N, lalu melakukan perulangan untuk mengisi array

dengan N buah data berat. Setelah itu, variabel min dan max diinisialisasi dengan nilai elemen pertama array sebagai pembanding awal. Selanjutnya dilakukan perulangan dari indeks ke-1 hingga N-1 untuk membandingkan setiap nilai: jika ditemukan nilai yang lebih kecil dari min, maka min diperbarui, dan jika lebih besar dari max, maka max juga diperbarui. Terakhir, program mencetak nilai minimum dan maksimum tersebut menggunakan fmt.Println. Program ini menggunakan metode linear search sehingga kompleksitas waktunya O(N).

2. Soal Unguided 2

Fakultas Informatika
School of Computing
Telkom University
informatics lab

2) Sebuah program digunakan untuk menentukan tarif ikan yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat ikan yang akan dijual.

**Masukan** terdiri dari dua baris, yang mana baris pertama terdiri dari dua bilangan bulat x dan y. Bilangan x menyatakan banyaknya ikan yang akan dijual, sedangkan y adalah banyaknya ikan yang akan dimasukan ke dalam wadah. Baris kedua terdiri dari sejumlah x bilangan riil yang menyatakan banyaknya ikan yang akan dijual.

**Keluaran** terdiri dari dua baris. Baris pertama adalah kumpulan bilangan riil yang menyatakan total berat ikan di setiap wadah (jumlah wadah tergantung pada nilai x dan y, urutan ikan yang dimasukan ke dalam wadah sesuai urutan pada masukan baris ke-2). Baris kedua adalah sebuah bilangan riil yang menyatakan berat rata-rata ikan di setiap wadah.

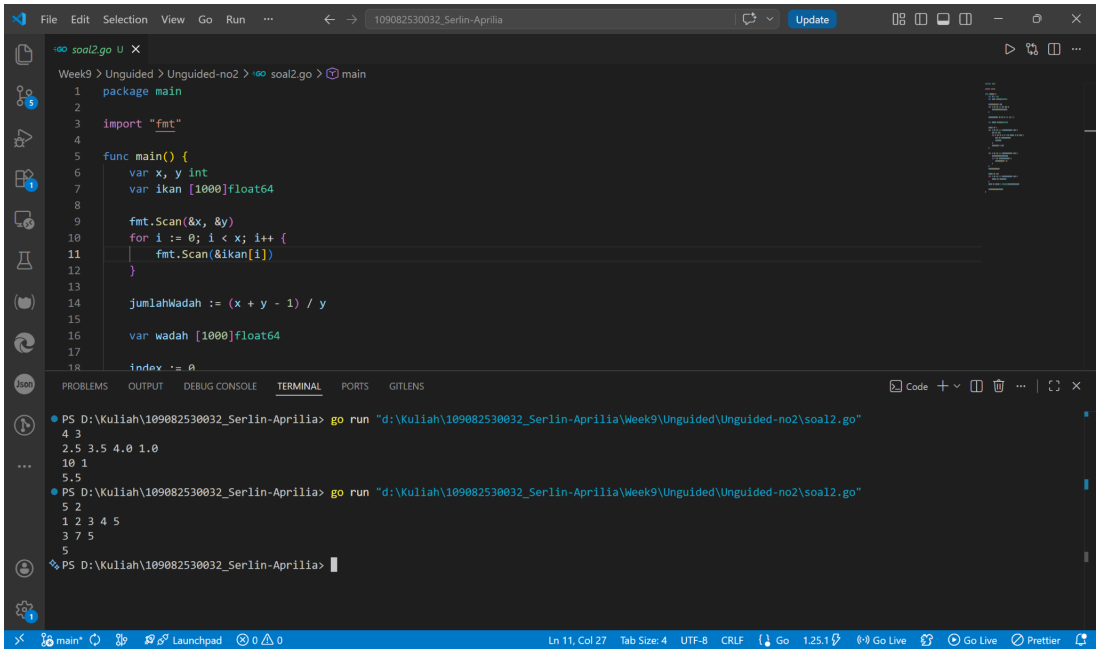
Jawaban :

```
package main


import "fmt"


func main() {
      var x, y int
      var ikan [1000]float64


      fmt.Scan(&x, &y)
```

```
	for i := 0; i < x; i++ {
		fmt.Scan(&ikan[i])
	}

	jumlahWadah := (x + y - 1) / y

	var wadah [1000]float64

	index := 0
	for i := 0; i < jumlahWadah; i++ {
		sum := 0.0
		for j := 0; j < y && index < x; j++ {
			sum += ikan[index]
			index++
		}
		wadah[i] = sum
	}

	for i := 0; i < jumlahWadah; i++ {
		fmt.Print(wadah[i])
		if i != jumlahWadah-1 {
			fmt.Print(" ")
		}
	}
	fmt.Println()

	total := 0.0
	for i := 0; i < jumlahWadah; i++ {
```

```
            total += wadah[i]

        }

        rata := total / float64(jumlahWadah)


        fmt.Println(rata)

}
```

Output :




Penjelasan :

Program tersebut merupakan implementasi pengolahan data berat ikan menggunakan array di bahasa Go, dengan tujuan mengelompokkan ikan ke dalam beberapa wadah dan menghitung total serta rata-rata beratnya. Pada awal program, variabel x dan y digunakan untuk menyimpan jumlah ikan dan kapasitas maksimal ikan per wadah, sedangkan array ikan bertipe float64 digunakan untuk menampung data berat ikan hingga maksimum 1000 elemen. Program kemudian membaca input jumlah ikan dan kapasitas wadah, lalu mengisi array dengan data berat ikan melalui perulangan. Nilai jumlahWadah dihitung menggunakan rumus (x + y - 1) / y untuk menentukan berapa banyak wadah yang dibutuhkan (pembulatan ke atas). Selanjutnya, dilakukan proses pengelompokan dengan iterasi menggunakan variabel

index untuk menjumlahkan berat ikan ke dalam setiap wadah hingga kapasitas y terpenuhi atau data habis, lalu hasilnya disimpan dalam array wadah. Setelah itu, program menampilkan total berat ikan pada setiap wadah dalam satu baris. Terakhir, program menghitung rata-rata berat per wadah dengan menjumlahkan seluruh isi array wadah kemudian membaginya dengan jumlah wadah, dan hasilnya ditampilkan sebagai output. Program ini menggunakan pendekatan iteratif sederhana dengan kompleksitas waktu O(n).

3. Soal Unguided 3

3) Pos Pelayanan Terpadu (posyandu) sebagai tempat pelayanan kesehatan perlu mencatat data berat balita (dalam kg). Petugas akan memasukkan data tersebut ke dalam array. Dari data yang diperoleh akan dicari berat balita terkecil, terbesar, dan reratanya.

Buatlah program dengan spesifikasi subprogram sebagai berikut:

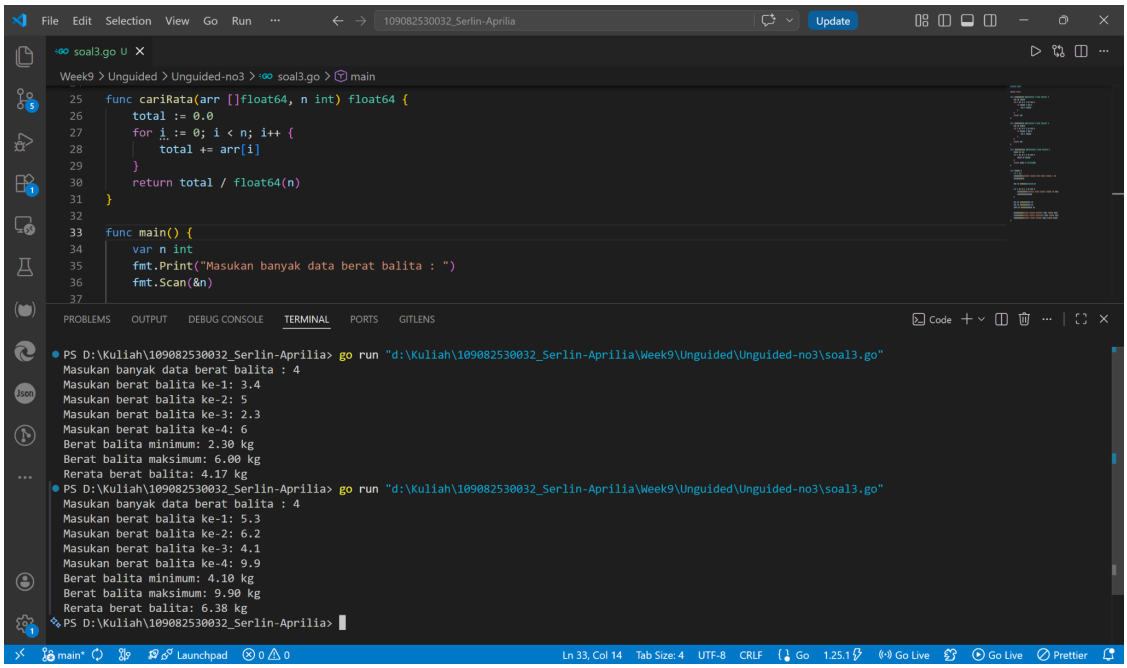
Jawaban :

```
package main


import "fmt"


func cariMin(arr []float64, n int) float64 {
	min := arr[0]
	for i := 1; i < n; i++ {
		if arr[i] < min {
			min = arr[i]
		}
	}
	return min
}
```

```
func cariMax(arr []float64, n int) float64 {
	max := arr[0]
	for i := 1; i < n; i++ {
		if arr[i] > max {
			max = arr[i]
		}
	}
	return max
}

func cariRata(arr []float64, n int) float64 {
	total := 0.0
	for i := 0; i < n; i++ {
		total += arr[i]
	}
	return total / float64(n)
}

func main() {
	var n int
	fmt.Print("Masukan banyak data berat balita : ")
	fmt.Scan(&n)

	arr := make([]float64, n)

	for i := 0; i < n; i++ {
		fmt.Printf("Masukan berat balita ke-%d: ", i+1)
		fmt.Scan(&arr[i])
```

```
        }


        min := cariMin(arr, n)
        max := cariMax(arr, n)
        rata := cariRata(arr, n)

        fmt.Printf("Berat balita minimum: %.2f kg\n", min)
        fmt.Printf("Berat balita maksimum: %.2f kg\n", max)
        fmt.Printf("Rerata berat balita: %.2f kg\n", rata)
}
```

Output :




Penjelasan :

Program tersebut digunakan untuk mengolah data berat balita dengan cara mencari nilai minimum, maksimum, dan rata-rata dari sejumlah data yang diinput oleh pengguna. Program dimulai dengan meminta jumlah data (n), kemudian pengguna memasukkan berat balita satu per satu yang disimpan dalam slice/array bertipe float64. Terdapat beberapa fungsi terpisah (subprogram), seperti cariMin, cariMax, dan cariRata, yang masing-masing

bertugas menghitung nilai terkecil, terbesar, dan rata-rata. Proses pencarian nilai minimum dan maksimum dilakukan dengan metode linear search, yaitu membandingkan setiap elemen dalam array secara berurutan, sedangkan rata-rata dihitung dengan menjumlahkan seluruh data lalu dibagi dengan jumlah elemen. Setelah semua perhitungan selesai, hasilnya ditampilkan ke layar dengan format yang rapi menggunakan fmt.Printf, termasuk pembulatan hingga dua angka di belakang koma serta satuan “kg”. Program ini menunjukkan penggunaan konsep dasar Go seperti fungsi, array/slice, perulangan, serta input-output.

D. Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa pencarian nilai minimum, maksimum, dan rata-rata merupakan bagian penting dalam pengolahan data numerik. Dengan memanfaatkan bahasa pemrograman Go, proses tersebut dapat diimplementasikan secara sederhana namun efektif menggunakan array atau slice sebagai media penyimpanan data. Penggunaan metode linear search memungkinkan program untuk menemukan nilai minimum dan maksimum dengan cara membandingkan setiap elemen secara berurutan, sedangkan perhitungan rata-rata dilakukan dengan menjumlahkan seluruh data kemudian dibagi dengan jumlah elemen.

Selain itu, penggunaan fungsi (subprogram) memberikan banyak keuntungan, seperti meningkatkan keterbacaan kode, mempermudah pengembangan program, serta memungkinkan penggunaan kembali kode (reusability). Program yang dibuat juga menunjukkan penerapan konsep dasar pemrograman seperti variabel, tipe data, perulangan, percabangan, dan input-output. Meskipun metode yang digunakan masih sederhana, pendekatan ini sudah cukup efektif untuk menangani data dalam jumlah kecil hingga menengah. Dengan demikian, praktikum ini memberikan pemahaman yang baik mengenai dasar algoritma pencarian dan pengolahan data serta bagaimana mengimplementasikannya dalam bahasa Go secara terstruktur dan sistematis.